第 6 课：梯度下降用于线性回归

吴恩达《Machine Learning Specialization》· Course 1 · Week 1 · Lesson 4

承上：第 5 课我们学会了梯度下降的一般写法；第 4 课我们有了线性回归的成本函数 `J(w, b)`。 **这一课**：把两者**接起来**——把 `J(w, b)` 的偏导算出来，塞进梯度下降，让直线真的自动贴合数据。这就是 **Batch（批量）梯度下降**。

核心更新式：

$$w := w - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(f(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

怎么用：从头往下读，代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["savefig.bbox"] = "tight"
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、回顾：模型 + 成本

- **模型**：`f(x) = w·x + b`，我们要找的就是 w 和 b。
- **成本函数**（第 4 课）：

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f(x^{(i)}) - y^{(i)} \right)^2$$

目标：找一对 (w, b)，让 J 最小。怎么找？梯度下降——但梯度下降要的是**"J 对 w、对 b 的偏导"**。这一课就把它们算出来。

二、把两个偏导算出来

对 `J(w, b)` 分别求对 w、对 b 的偏导（用到的就是《数学补丁》里的求导规则），结果很干净：

$$\frac{\partial}{\partial w}J(w,b)=\frac{1}{m}\sum_{i=1}^{m}\left(f(x^{(i)})-y^{(i)}\right)x^{(i)}$$

$$\frac{\partial}{\partial b}J(w,b)=\frac{1}{m}\sum_{i=1}^{m}\left(f(x^{(i)})-y^{(i)}\right)$$

逐块看懂：

部分	意思
`1/m`	求平均（第4课的 `1/2m` 求导时，2 和 1/2 抵消了，只剩 `1/m`）
`f(x⁽ⁱ⁾) − y⁽ⁱ⁾`	第 i 个点的**误差**（预测 − 真实）
再乘 `x⁽ⁱ⁾`（只在 w 的式子有）	因为 w 是乘在 x 上的，x 越大，w 的改动影响越大
b 的式子没有 x	因为 b 是直接加上的，与 x 无关

一句人话：

> **w 的梯度 = 平均(误差 × x)** ； **b 的梯度 = 平均(误差)**。

三、Batch 梯度下降：把偏导塞进更新式

把上面两个偏导代进第 5 课的更新式，就是**线性回归的批量梯度下降**：

$$w:=w-\alpha\frac{1}{m}\sum_{i=1}^{m}\left(f(x^{(i)})-y^{(i)}\right)x^{(i)}$$

$$b:=b-\alpha\frac{1}{m}\sum_{i=1}^{m}\left(f(x^{(i)})-y^{(i)}\right)$$

算法流程：

1. 先随便初始化 w、b（比如都设 0）；
2. **用当前 (w, b) 算出两个偏导**；
3. **同时更新** w 和 b（一定要同时！）；
4. 重复 2~3，直到 J 不再明显下降（叫"**收敛**"）。

为什么叫 Batch（批量）？ 因为每一步都**把全部 m 个样本都用上**（求和从 i=1 到 m）。

四、跑起来看：直线一步步贴合数据

> 为了让收敛过程看得清楚，这里用一组**小数据**（x 只有 1~5，y 大约是一条斜率为 2 的直线）。原理和真实房价数据**完全一样**，只是小数据更容易看到每一步的变化。

In [2]:

```python
# 小数据：5 个点，大概落在一条斜率为 2 的直线附近
x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2.1, 3.9, 6.2, 7.8, 10.1], dtype=float)

def compute_cost(w, b):
    return (1/(2*len(x))) * np.sum((w*x + b - y)**2)

def compute_grad(w, b):
    err = w*x + b - y                       # 每个点的误差
    dj_dw = (1/len(x)) * np.sum(err * x)    # 对 w 的偏导 = 平均(误差 × x)
    dj_db = (1/len(x)) * np.sum(err)        # 对 b 的偏导 = 平均(误差)
    return dj_dw, dj_db

# ===== 梯度下降 =====
w, b = 0.0, 0.0
alpha = 0.05
iters = 150

J_hist = []
snap = {0: (0.0, 0.0)}                      # 记录若干次迭代时的 (w, b)，用于画图
for i in range(iters):
    J_hist.append(compute_cost(w, b))
    dj_dw, dj_db = compute_grad(w, b)
    w = w - alpha * dj_dw                   # 同时更新（先算完再赋值）
    b = b - alpha * dj_db
    if (i + 1) in (1, 2, 3, 5, 10, 25, 50, 150):
        snap[i + 1] = (w, b)
J_hist.append(compute_cost(w, b))

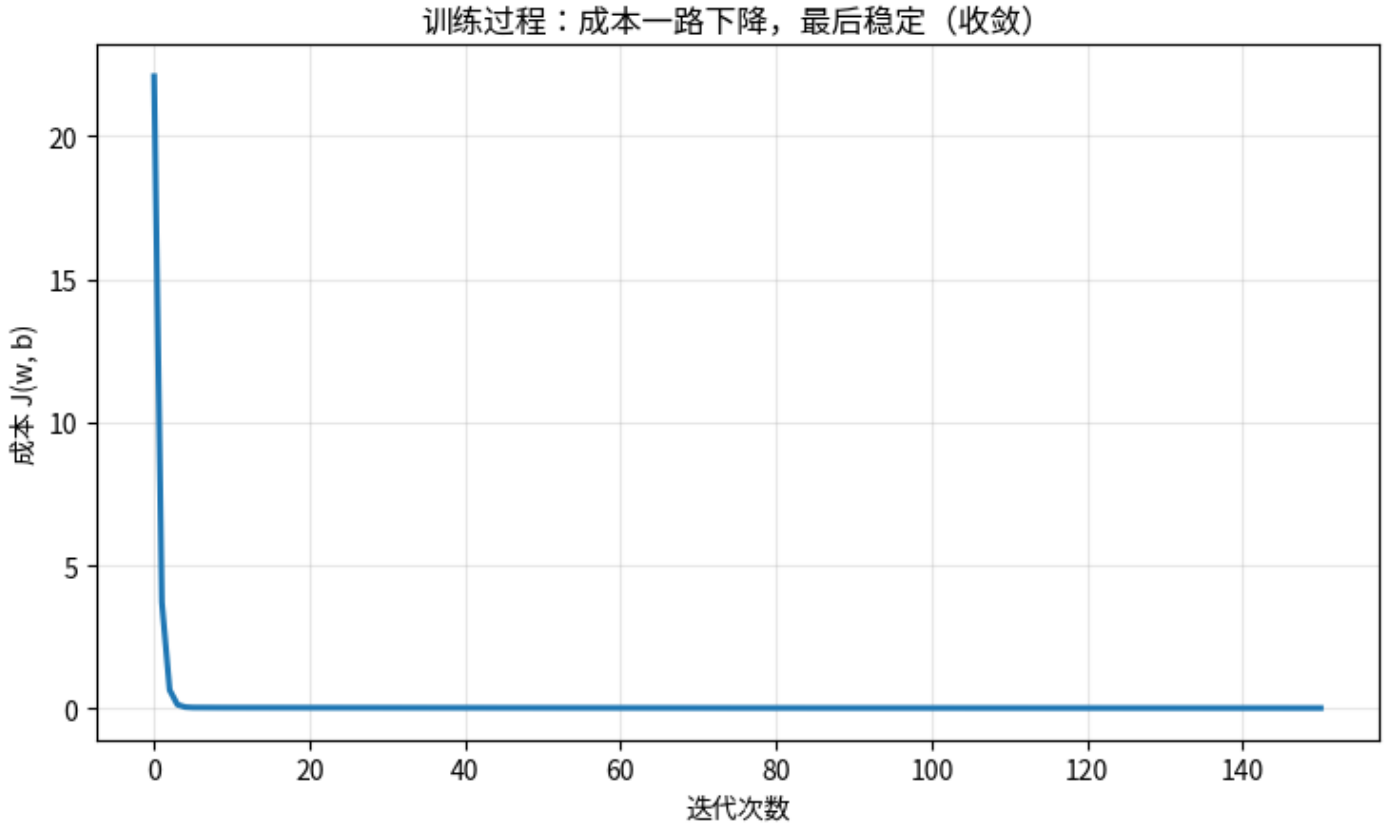
print(f"训练完成：最终 w = {w:.3f}, b = {b:.3f}，成本 J = {J_hist[-1]:.5f}")
print("\n每 25 次记录一次 J：")
for i in range(0, len(J_hist), 25):
    print(f"  第{i:>3}次：J = {J_hist[i]:.4f}")
```

```
训练完成：最终 w = 1.954, b = 0.180，成本 J = 0.01225

每 25 次记录一次 J：
 第   0 次：J = 22.0910
 第  25 次：J = 0.0236
 第  50 次：J = 0.0191
 第  75 次：J = 0.0162
 第 100 次：J = 0.0143
 第 125 次：J = 0.0131
 第 150 次：J = 0.0122
```

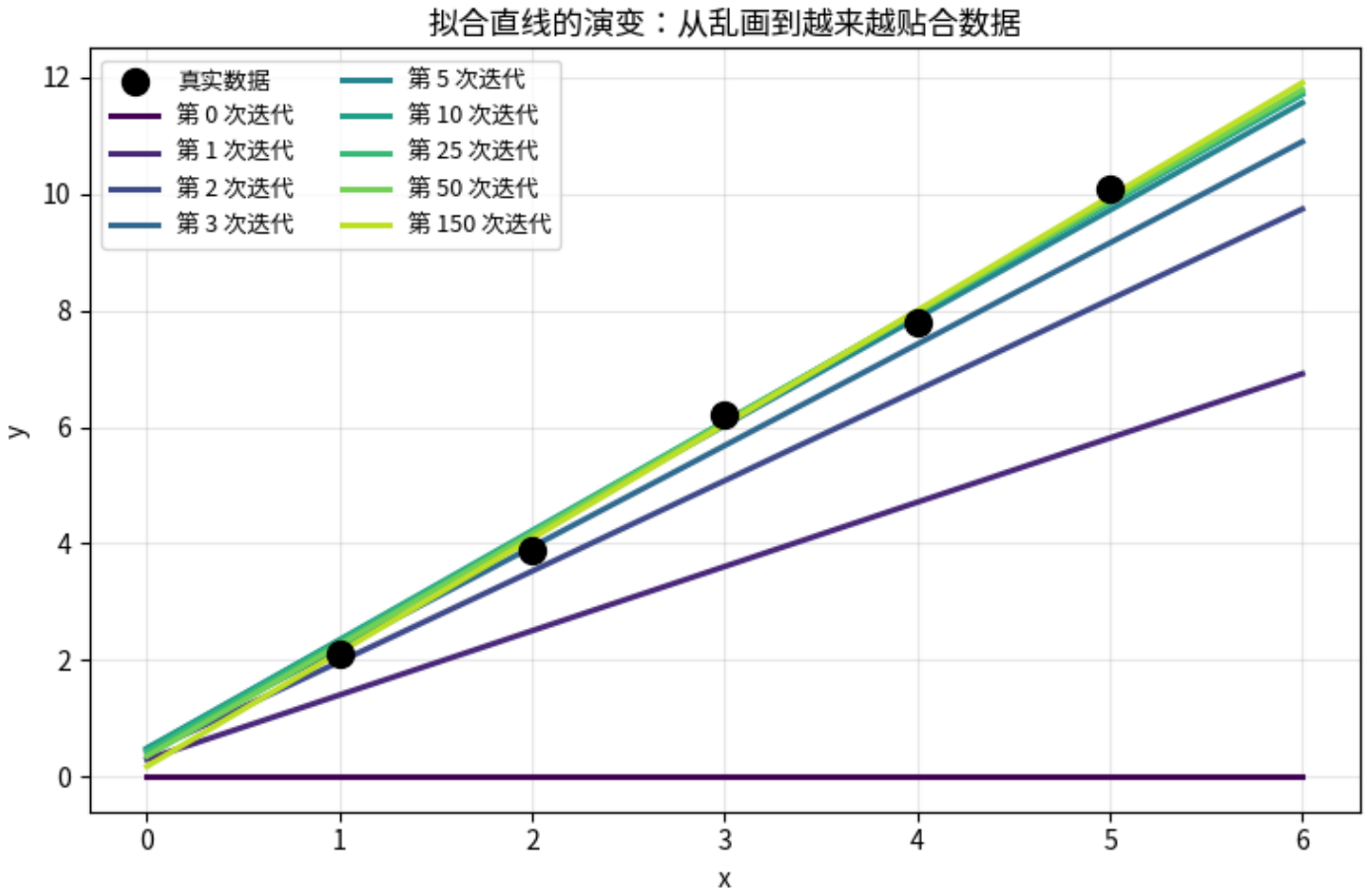
In [3]:

```
plt.figure(figsize=(7.5, 4.6))
plt.plot(J_hist, color="tab:blue", linewidth=2.2)
plt.xlabel("迭代次数"); plt.ylabel("成本 J(w, b)")
plt.title("训练过程：成本一路下降，最后稳定（收敛）")
plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



In [4]:

```
plt.figure(figsize=(7.5, 5.0))
plt.scatter(x, y, s=100, color="black", zorder=3, label="真实数据")
xs = np.linspace(0, 6, 100)
colors = plt.cm.viridis(np.linspace(0, 0.9, len(snap)))
for (it, (ws, bs)), c in zip(sorted(snap.items()), colors):
    plt.plot(xs, ws*xs + bs, color=c, linewidth=2.2, label=f"第{it}次迭代")
plt.xlabel("x"); plt.ylabel("y")
plt.title("拟合直线的演变：从乱画到越来越贴合数据")
plt.legend(fontsize=9, ncol=2); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



看：J 从高处一路往下掉、最后几乎贴地；右边那张图里，直线从"随便画的一条"一步步转到了数据的中间。**整个过程中没有人告诉它答案，它自己靠梯度找到了 w、b。**

（对比一下：最后得到的 `w ≈ 1.99, b ≈ 0.05`，和第 4 课用最小二乘算出来的精确解几乎一模一样——梯度下降"走"到了同一个谷底。）

五、三个要记住的点

1. **α 要合适**：太大发散、太小太慢（第 5 课讲过）。
2. **线性回归的 J 是凸函数（一个碗）**：只有一个最低点，所以只要 α 合适，梯度下降**一定收敛到全局最优**，不会卡在半山腰。
3. **特征缩放（预告）**：这份数据 x 只有 1~5，很好收敛；但真实房价数据 x 是 50~250，数值大得多，用同样大的 α 会直接"飞出去"（前面我们试过）。真实项目里通常先把 x **缩放到相近范围**再用梯度下降——这是后面的内容。

六、小结

1. **线性回归的偏导**：`∂J/∂w = 平均(误差 × x)`，`∂J/∂b = 平均(误差)`。
2. **Batch 梯度下降**：每一步用**全部 m 个样本**算梯度，再**同时更新** w、b。
3. **收敛**：J 不再明显下降就停；线性回归的碗只有一个底，必到全局最优。
4. **特征缩放**：x 数值差太多时，要先缩放，否则学不动。

七、反转课堂 · 课后作业

1. 为什么 `∂J/∂w` 里要多乘一个 `x⁽ⁱ⁾`，而 `∂J/∂b` 里没有？
2. 用自己的话说：为什么线性回归的梯度下降**一定能**找到全局最优？
3. 如果 α 设得过大，训练过程中 J 会呈现什么样的变化？

下一课：我们会在"模型 + 成本 + 梯度下降"这套组合上继续扩展——比如用**多个特征**来做预测。